שמות הסטודנטים	ת.ז
דניאל ג'נודי	*318852571*
הראל מזרחי	*322809922*
קוד פרויקט	*051*
מנחה	ד"ר אריאל רוט

Abstract

This project implements SeekSuit, an AI-powered product discovery and showcase platform tailored for Janudi Fashion. The system integrates a hybrid visual-textual search engine, an automated image processing pipeline that transforms raw phone photos into studio-quality catalog listings, a Virtual Try-On (VTO) feature powered by FitDiT running on RunPod serverless GPU infrastructure, and an autonomous LLM-based agent that analyzes inventory, search trends, and product engagement to generate actionable business insights. The platform also includes a content management interface that enables the store owner to update site texts, gallery images, and catalog data without technical assistance. The solution enhances product discovery, catalog digitization efficiency, and data-driven business strategy.

---

1. Introduction

The men's fashion retail domain includes a wide variety of styles, cuts, and colors, which makes product discovery challenging, especially for businesses that hold large physical stock without a complete digital catalog. SeekSuit is a web-based platform built for Janudi Fashion to bridge this gap by enabling customers to discover products through AI-powered visual and textual search, and enabling store managers to manage the catalog, process product images, and showcase garments on virtual models , eliminating the need for physical photoshoots with models. The platform is designed as a showcase and discovery tool rather than a traditional inventory management system: it presents the store's existing collection and drives in-store visits.

2. Problem Statement

Following a transition from wholesale to retail, Janudi Fashion manages a vast physical inventory that requires a complete digital catalog. Producing such a catalog traditionally requires professional studio photography , including photoshoots with live models , a process that is prohibitively expensive and slow at the required scale. At the same time, the sheer volume and diversity of items makes it difficult for customers to browse efficiently and locate specific styles. This leads to poor product discovery and missed sales opportunities. In parallel, the business lacks real-time visibility into inventory gaps, catalog completeness, and evolving customer demand, which makes stock and marketing decisions largely reactive.

This project addresses these challenges by introducing a cost-effective mobile digitization workflow with automated image enhancement, a Virtual Try-On system that enables showcasing garments on virtual models without physical photoshoots, and an AI-powered multimodal search using images and natural language. Additionally, an autonomous insight agent continuously analyzes inventory status, image coverage, search trends, and product engagement to generate actionable business recommendations.

---

3. Objectives

1. Develop a Hybrid Search Engine: *Implement a dual-mode search system enabling users to locate products via visual similarity and semantic natural language queries.*

2. Streamline Catalog Digitization: *Implement an automated image processing pipeline that enables store managers to upload raw smartphone photos and automatically generate studio-quality catalog listings through background removal, lighting correction, and canvas normalization.*

3. Enable Virtual Try-On: *Integrate a Virtual Try-On feature that allows store managers to showcase garments on virtual models, eliminating the need for physical photoshoots with live models.*

4. Deploy an Autonomous Insight Agent: *Integrate an LLM-based agent that analyzes inventory status, image coverage, search trends, and product engagement data to provide strategic business recommendations to store managers.*

5. Design for Administrator Control: *Architect the platform to maximize store manager autonomy , enabling dynamic updates to site content, product catalog, gallery, and visual assets through the admin interface, without requiring developer intervention.*

6. Architect a Full-Stack Solution: *Build a robust client-side interface for product showcase and search, backed by a server-side system that manages the insight agent, executes the image processing pipeline, and maintains the image repository.*

4. User Stories

Customer

1. Visual Image Search *(KAN-88, KAN-202, KAN-204):*
As a customer, I want to upload a reference photo so I can find visually similar suits in the catalog.

2. Natural Language Search *(KAN-88):*
As a customer, I want to search using natural language descriptions so I can find suits based on context rather than exact keywords.

3. Similar Product Recommendations *(KAN-88, KAN-75):*
As a customer, I want to see visually similar product recommendations when viewing a product, so I can explore alternatives.

4. Catalog Filtering *(KAN-25, KAN-190):*
As a customer, I want to filter products by color and type so I can narrow down the catalog using standard attributes.

5. Contact Request *(KAN-193):*
As a customer, I want to submit a contact request so a store representative can follow up with me personally.

6. Store Information *(KAN-193):*
As a customer, I want to view store information such as location and opening hours so I can plan my visit.

Store Manager (Admin)

7. Automated Image Processing *(KAN-119, KAN-120, KAN-123, KAN-200, KAN-203):*
As a store manager, I want to upload raw smartphone photos and have the system automatically generate studio-quality images, so I can build the catalog without professional photography.

8. Virtual Try-On *(KAN-204, KAN-205, KAN-206, KAN-207, KAN-208, KAN-209, KAN-210, KAN-211, KAN-222):*
As a store manager, I want to use Virtual Try-On to generate images of garments on virtual models, so I can showcase products without physical photoshoots.

9. AI Business Insights *(KAN-98):*
As a store manager, I want to receive AI-generated business insights based on inventory and search data, so I can make informed decisions about stock and marketing.

10. Catalog Management *(KAN-19, KAN-78):*
As a store manager, I want to manage the product catalog , adding, editing, and removing items , through a secure admin interface.

11. Site Content Management *(KAN-218):*
As a store manager, I want to update site texts, gallery images, and store content dynamically so the website stays current without developer involvement.

12. Secure Admin Login *(KAN-46):*
As a store manager, I want to securely log in to the admin dashboard so that sensitive management features are protected from unauthorized access.

Sample Code

User Story 1: Visual Image Search

AIService/image_search.py , CLIP image embedding

```python
def embed_image(image_bytes: bytes) -> list[float]:
    """
    Embed raw image bytes with CLIP.
    Returns a unit-normalized list of 512 floats suitable for cosine similarity.
    """
    model, processor = _get_clip()
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    inputs = processor(images=image, return_tensors="pt")
    with torch.no_grad():
        features = model.get_image_features(**inputs)
        features = features / features.norm(dim=-1, keepdim=True)
    return features[0].tolist()
```

Backend/src/controllers/search.controller.ts , pgvector similarity query

```typescript
// POST /api/search/image
// Embeds the uploaded image with CLIP and finds the most visually similar products
// using cosine distance (<=> operator) via pgvector.
export const searchByImage = async (req: Request, res: Response) => {
  const { embedding, dominantColor } = await aiService.embedImage(
    file.buffer, file.originalname, {}
  );
  const pgVector = `[${embedding.join(',')}]`;

  const rows = await prisma.$queryRaw<SearchResult[]>`
    WITH scored AS (
      SELECT DISTINCT ON (p.id)
        p.id, p.name, p.sku, p.type::text, p.color, p.status::text,
        1 - (pi.embedding <=> ${pgVector}::vector) AS similarity
      FROM "ProductImage" pi
      JOIN "Product" p ON pi."productId" = p.id
      WHERE pi.embedding IS NOT NULL
      ORDER BY p.id, pi.embedding <=> ${pgVector}::vector ASC
    )
    SELECT * FROM scored ORDER BY similarity DESC LIMIT ${limit}
  `;
  res.json({ results: rows, dominantColor });
};
```

User Story 2: Natural Language Search

AIService/image_search.py , Hebrew-aware CLIP text embedding

```python
def embed_text(text: str) -> list[float]:
    """
    Embed a text query with CLIP's text encoder.
    For Hebrew queries, translates word-by-word using FASHION_SYNONYMS and
    applies prompt ensembling across synonym variants for a more robust vector.
    """
    query = text.strip()
    if not _contains_hebrew(query):
        return _encode_texts_averaged([query])

    words = query.split()
    translated, ensemble_pos = [], None
    for word in words:
        if word in FASHION_SYNONYMS:
            syns = FASHION_SYNONYMS[word]      # e.g. "חליפה" → ["suit jacket", "men's blazer", ...]
            translated.append(syns[0])
            if ensemble_pos is None and len(syns) > 1:
                ensemble_pos = (len(translated) - 1, syns)
        else:
            translated.append(word)

    base = ' '.join(translated)
    variants = [base]
    if ensemble_pos:
        pos, syns = ensemble_pos
        for syn in syns[1:]:
            v = translated.copy(); v[pos] = syn
            variants.append(' '.join(v))

    return _encode_texts_averaged(variants)   # average + renormalize
```

Backend/src/controllers/search.controller.ts - text search with color filter

```typescript
// POST /api/search/text
export const searchByText = async (req: Request, res: Response) => {
  const { embedding } = await aiService.embedText(query.trim());
  const pgVector = `[${embedding.join(',')}]`;

  const rows = await prisma.$queryRaw<SearchResult[]>`
    WITH scored AS (
      SELECT DISTINCT ON (p.id) p.id, p.name, p.color, p."dominantColor",
        1 - (pi.embedding <=> ${pgVector}::vector) AS similarity
      FROM "ProductImage" pi JOIN "Product" p ON pi."productId" = p.id
      WHERE pi.embedding IS NOT NULL
      ORDER BY p.id, pi.embedding <=> ${pgVector}::vector ASC
    )
    SELECT * FROM scored ORDER BY similarity DESC LIMIT ${limit}
  `;

  // Hard-filter by color family when a color word is detected in the query
  const detectedColor = detectQueryColor(query.trim());
  if (detectedColor) {
    const filtered = results.filter(r =>
      productMatchesColorFamily(detectedColor, r.dominantColor, r.color)
    );
    if (filtered.length > 0) results = filtered;
  }
  res.json({ results });
};
```

User Story 8: Virtual Try-On

Backend/src/services/vto.service.ts , RunPod serverless dispatch

```
// Send a VTO job to RunPod serverless GPU endpoint (FitDiT model).
// Returns a job ID used for polling status.
async function runpodRun(input: Record<string, unknown>): Promise<string> {
  const res = await fetch(`${RUNPOD_BASE}/run`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${RUNPOD_API_KEY}`,
      'Content-Type':  'application/json',
    },
    body: JSON.stringify({ input }),
  });
  if (!res.ok) throw new Error(`RunPod /run failed: ${res.status}`);
  const data = await res.json() as { id: string };
  return data.id;
}

// Trigger a VTO job for a product image.
export async function triggerVTOJob(productId: string, sourceImageId: string) {
  const sourceImage = await prisma.productImage.findUnique({ where: { id: sourceImageId } });
  const product     = await prisma.product.findUnique({ where: { id: productId } });
  const garmentType = product?.type === 'VEST' ? 'VESTS' : 'JACKETS';

  const runpodJobId = await runpodRun({
    garment_url:     sourceImage!.processedUrl,
    garment_type:    garmentType,
    product_id:      productId,
    source_image_id: sourceImageId,
  });

  return prisma.vTOJob.update({
    where: { id: job.id },
    data:  { runpodJobId, status: 'RUNNING' },
  });
}
```

User Story 9: AI Business Insights

Backend/src/services/insights.service.ts , LLM agent with tools

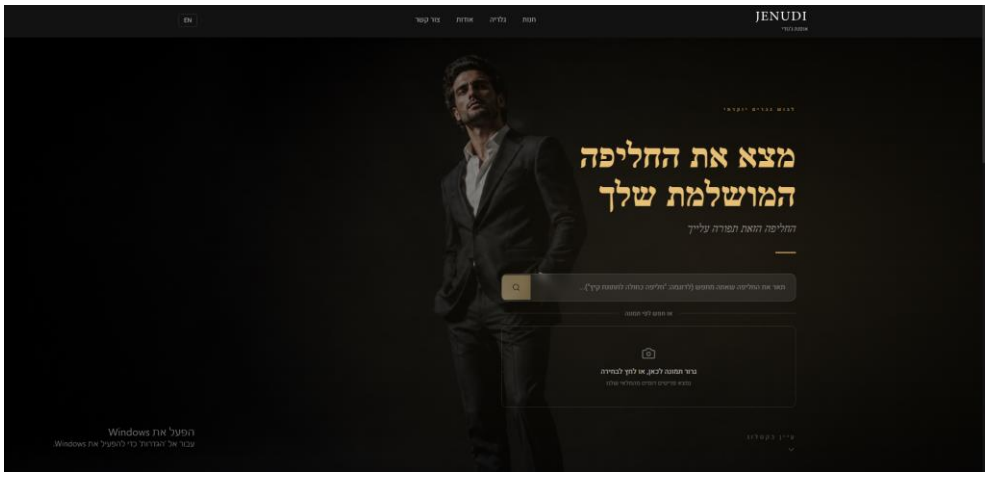
```
// Tools the agent can call to query real store data
const TOOL_DEFINITIONS: ToolDefinition[] = [
  { name: 'getInventoryOverview',
    description: 'Returns total product count broken down by type and stock status.' },
  { name: 'getSearchTrends',
    description: 'Returns top searched queries, colors, and zero-result queries from the past N days.' },
  { name: 'getStockGapFromSearch',
    description: 'Identifies colors searched frequently but with zero results — unmet demand.' },
  { name: 'getProductViewTrends',
    description: 'Returns most-viewed products by weighted score over the past N days.' },
  // + 3 more tools (getStockDetails, getColorDistribution, getImageCoverage)
];

// Run the agent: LLM decides which tools to call, receives data, returns structured insights
export async function getAutoInsights(): Promise<Insight[]> {
  const llm = createLLMProvider();   // Google Gemini (default) or Anthropic Claude
  const raw = await llm.runWithTools(
    AUTO_SYSTEM_PROMPT,
    'Analyze the store data and generate business insights.',
    TOOL_DEFINITIONS,
    handleToolCall,    // dispatches tool calls to real DB queries
  );
  const match = raw.match(/\[[\s\S]*\]/);
  return JSON.parse(match![0]) as Insight[];
}
```

5. Application Screenshots

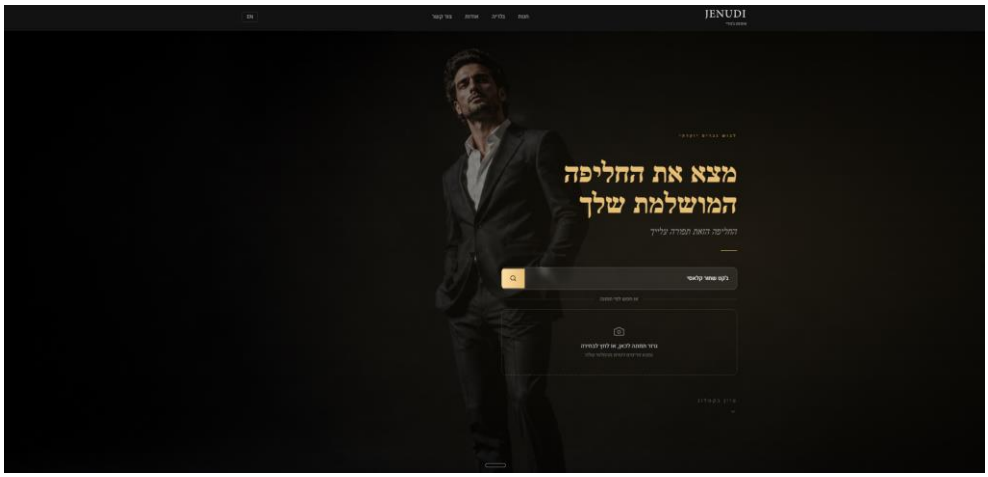
Customer Interface

Figure 1: Home Page: Hero Section & AI Search



The landing page presents the store brand alongside the dual-mode AI search widget. Customers can type a natural language query or upload a reference image to find visually similar products.

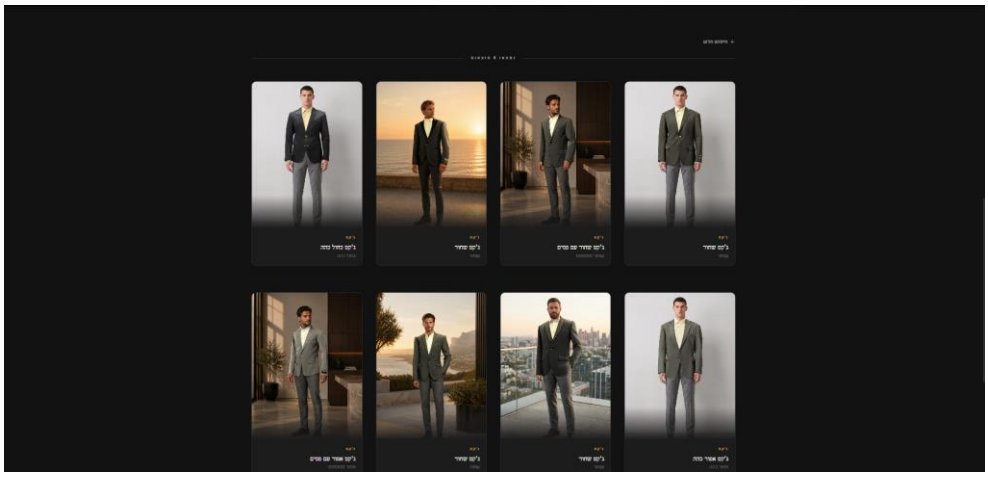
Figure 2: Text Search: Query Entry



A customer has entered a Hebrew text query. The system passes the query through a Hebrew-aware CLIP text encoder before executing a pgvector similarity search

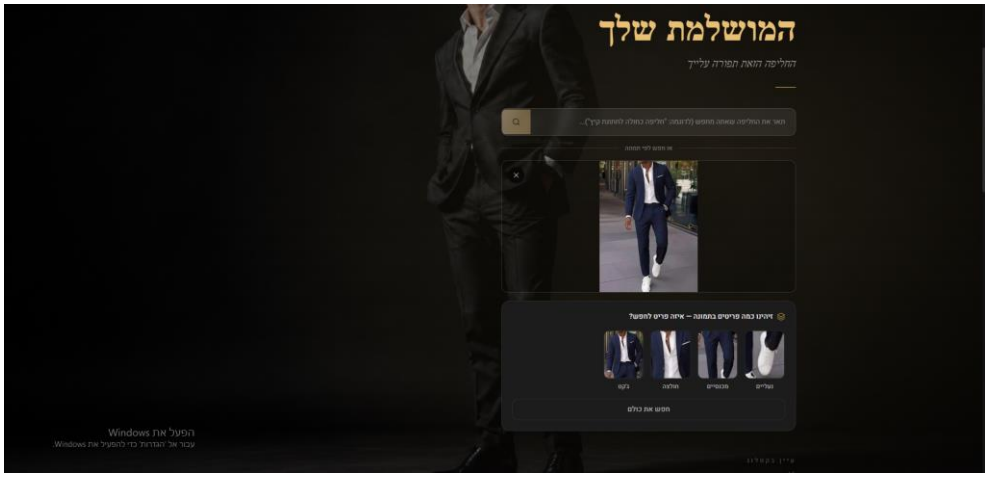
.

Figure 3: Text Search: Results Grid



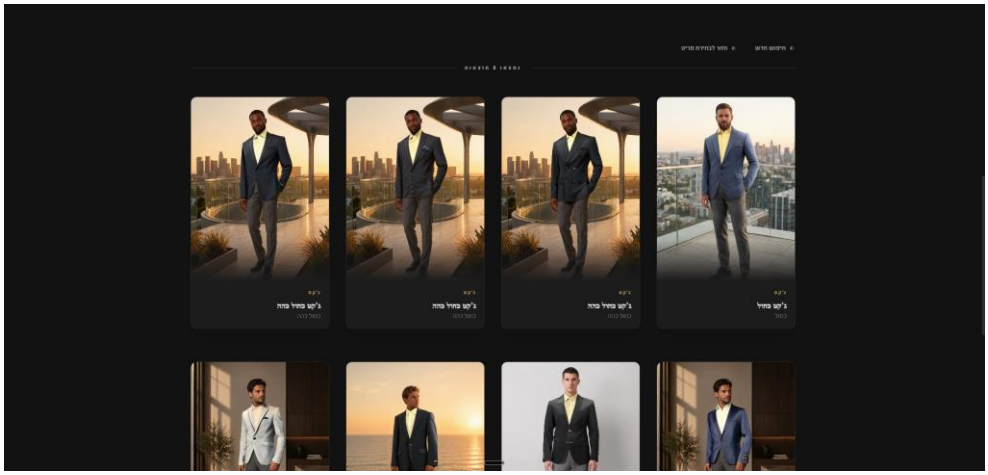
Matching products are displayed as a grid of cards. Results are ranked by CLIP cosine similarity and filtered by detected color and product type extracted from the query.

Figure 4: Image Search: Photo Upload & Item Detection



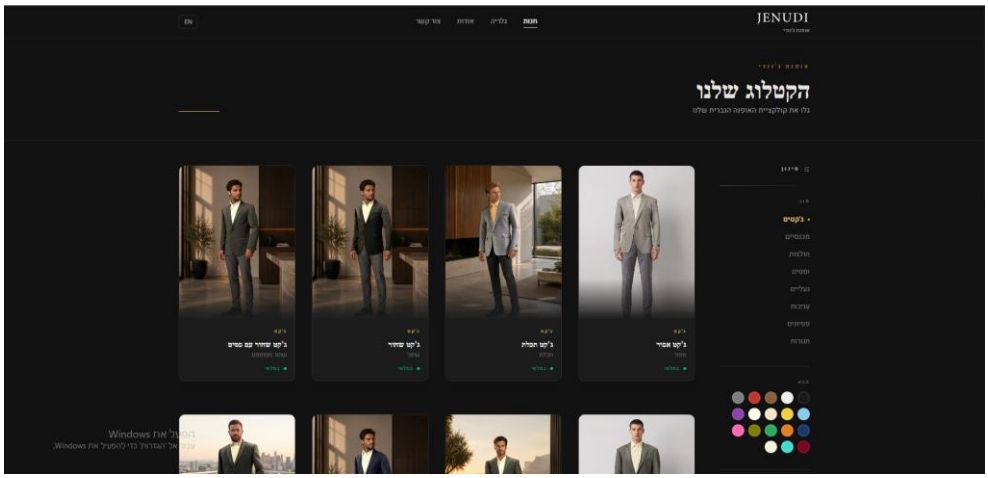
A customer uploads a reference outfit photo. The system runs YOLOS clothing detectionand presents an item picker when multiple garment types are detected.

Figure 5: Image Search: Results Grid



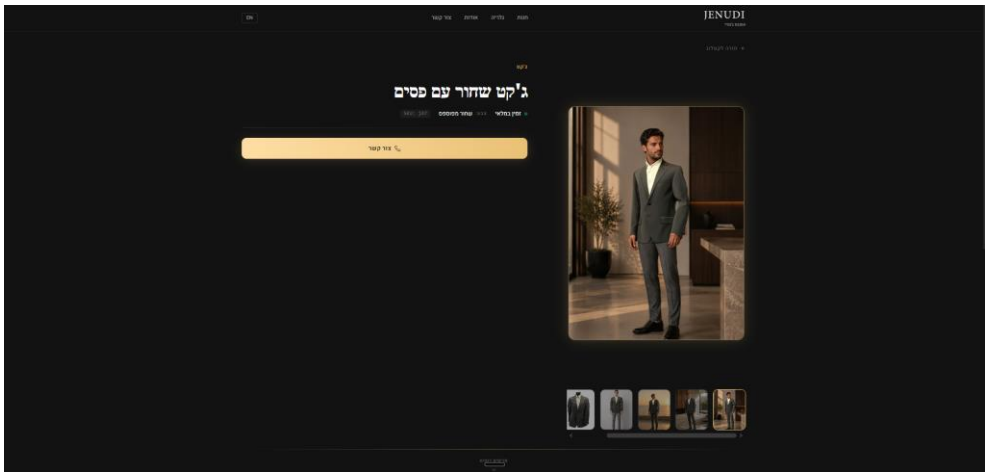
Products visually similar to the uploaded image are returned, ranked by CLIP image-to-image cosine similarity with a color-proximity boost applied.

Figure 6: Product Catalog: Browse & Filter



The public catalog page displays all available products in a grid layout. Customers can filter by product type and color using the panel on the right.

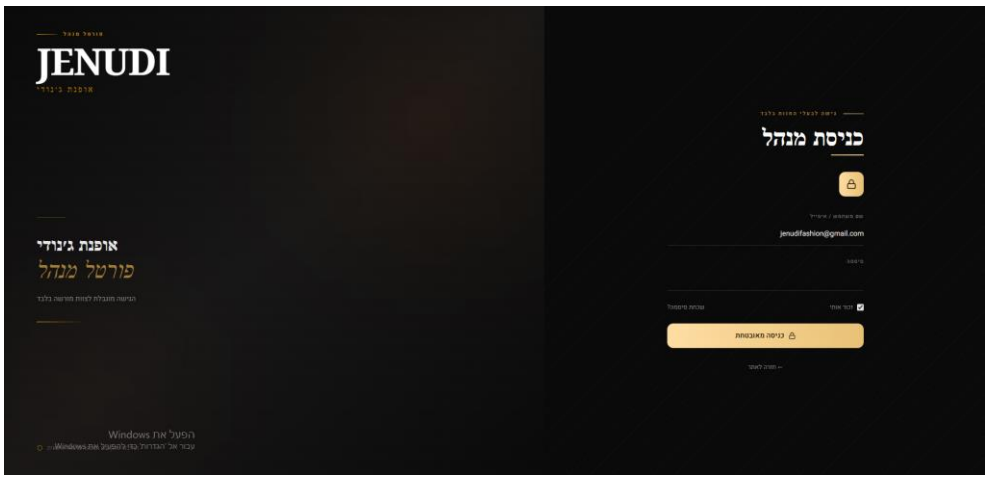
Figure 7: Product Detail Page



The product detail page shows the full product name, type, color, SKU, and a gallery of Virtual Try-On images. A contact button enables customers to reach the store directly.

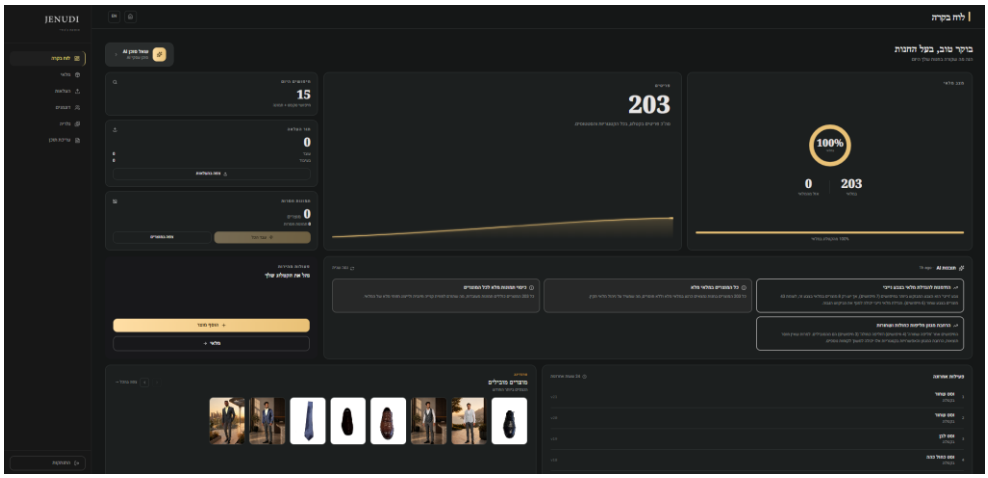
Admin Interface

Figure 8: Admin Login



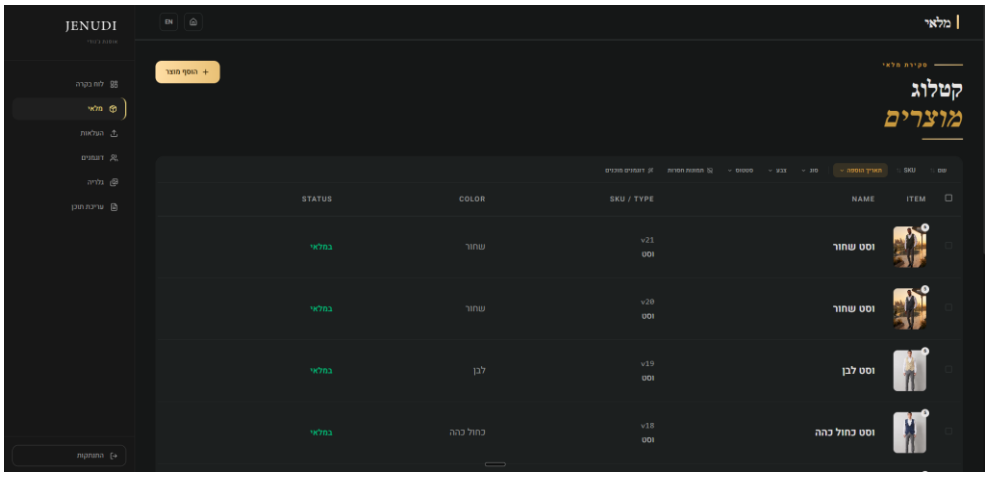
Secure authentication screen powered by Supabase Auth. Access to all admin features is restricted to authorized store managers.

Figure 9: Admin Dashboard: AI Business Insights



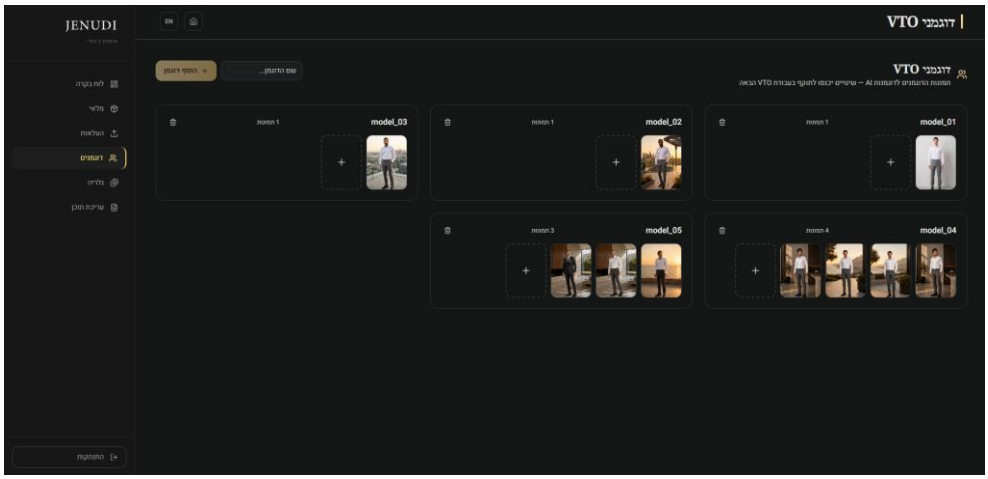
The admin dashboard displays key inventory metrics alongside AI-generated business insights. The autonomous LLM agent queries inventory status, search trends, and engagement data to produce actionable recommendations.

Figure 10: Product Catalog Management



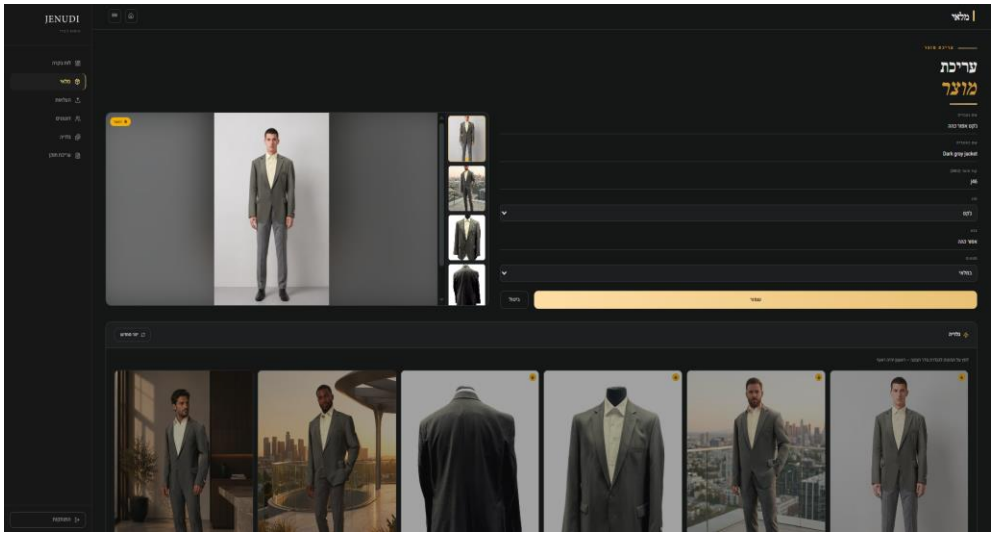
The admin product catalog lists all items with their SKU, type, color, stock status, and image count. Admins can add, edit, or remove products and apply filters to manage large catalogs efficiently.

Figure 11: Virtual Try-On (VTO) Model Management



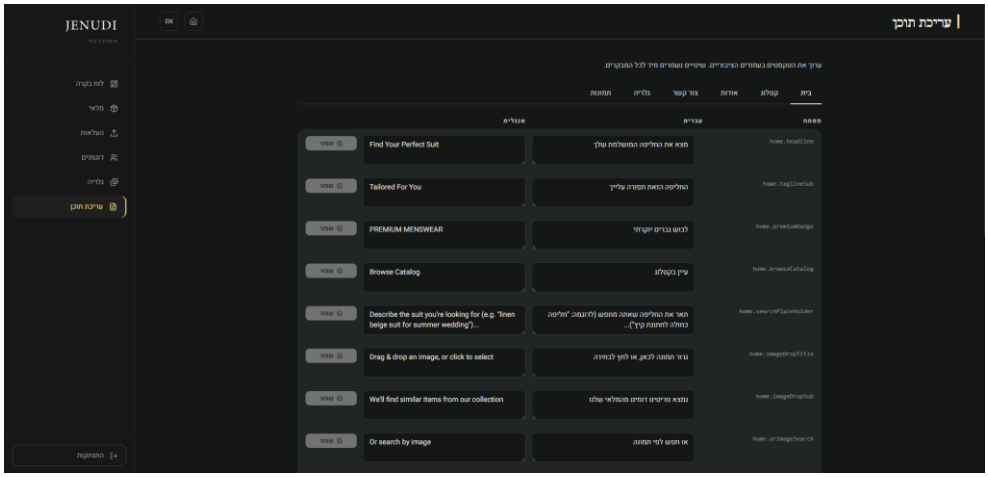
The VTO models management page displays all virtual model profiles. Each model card shows generated try-on images. Admins can trigger new FitDiT jobs directly from this interface.

Figure 12: Virtual Try-On Management



The admin Virtual Try-On interface enables assigning processed garment images to model photos and triggering FitDiT (via RunPod) to generate composited try-on images. Results appear in the product gallery alongside the original processed shots, and admins can reorder or regenerate them directly from this view.

Figure 13: Site Content Editor



The bilingual content editor allows store managers to update all public-facing site texts in Hebrew and English simultaneously. Changes are saved instantly and reflected on the live site.

6. Typical User Flows

1. Customer Visual Search: *Open homepage → Click Camera icon in search bar → Upload a reference photo → System detects clothing items (item picker) → Select specific item → View visually similar products → Click product to view details*

2. Customer Natural Language Search: *Open homepage → Type a descriptive query (e.g. "blue suit") → Press Enter → View results filtered by semantic similarity → Refine using color/type filters → Click product to view details*

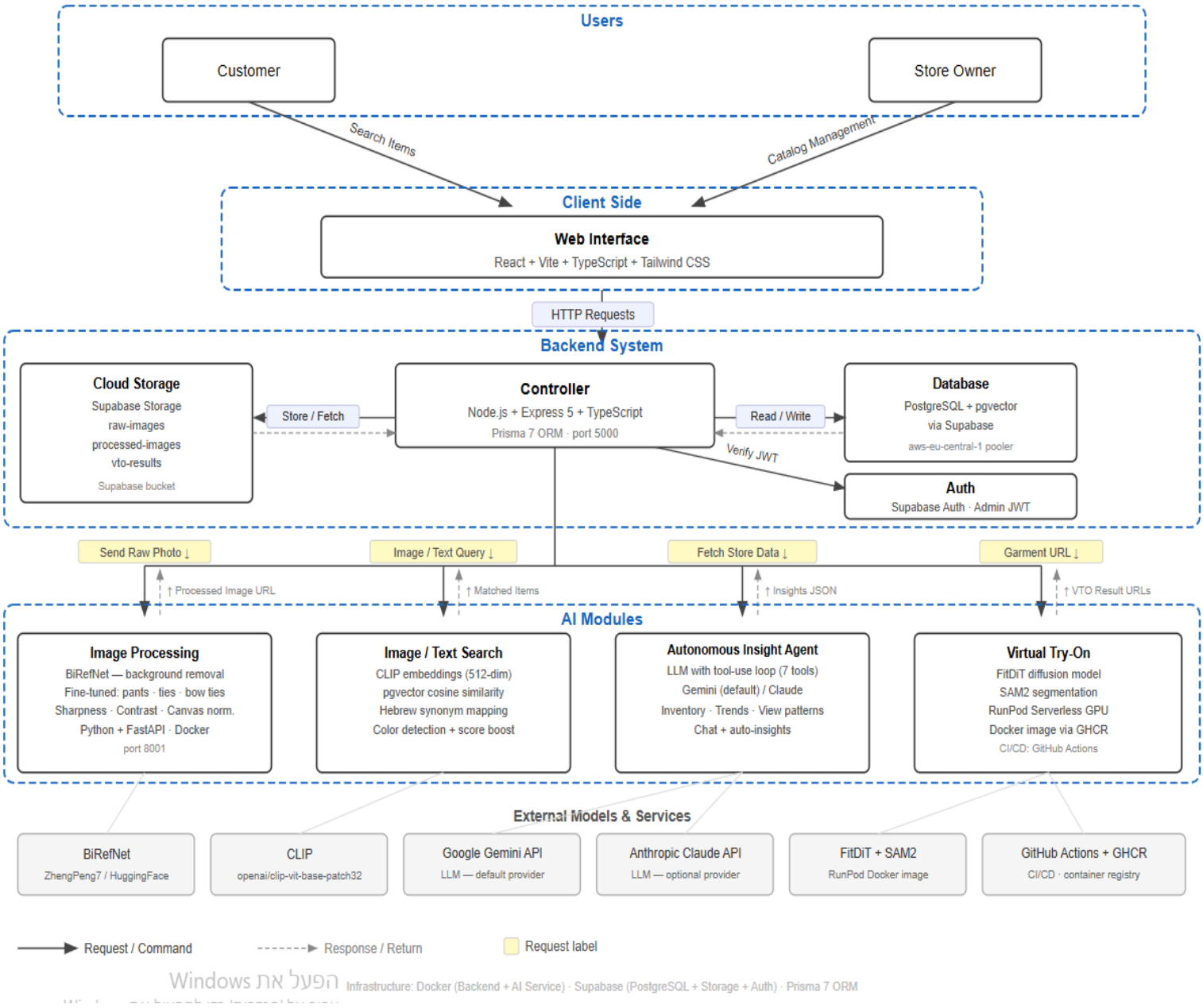
3. Admin , Image Processing: *Log in to admin dashboard → Navigate to Uploads page → Drag and drop raw smartphone photos → System queues and processes each image (background removal, enhancement, canvas normalization) → Processed images appear in product gallery*

4. Admin , Virtual Try-On: *Log in to admin dashboard → Open a product → Select a processed image → Click "Dress Models" → System sends garment to RunPod (FitDiT model) → View results on virtual models → Select preferred images → Publish to product gallery*

5. Admin , AI Business Insights: *Log in to admin dashboard → Navigate to Insights → System automatically queries inventory, search trends, and engagement data → LLM agent generates 3-6 actionable insights → Admin can also chat with the agent for custom queries*

6. Admin , Content Management: *Log in to admin dashboard → Navigate to Content Editor → Edit site texts (Hebrew/English), gallery images, or store info → Changes reflect immediately on the live site*

7. Project Entity Diagram



8. Implementation

8.1 Technologies Used

Frontend

Technology	Purpose
React	Component-based UI framework
Vite	Development server and production bundler
TypeScript	Static typing across all frontend code
Tailwind CSS	Utility-first styling with custom design tokens
React Router	Client-side routing (SPA navigation)
Axios	HTTP client for API requests with timeout control
Lucide React	Icon library

Backend

Technology	Purpose
Node.js	Server runtime
Express	HTTP server and routing framework
TypeScript	Static typing
Prisma ORM	Type-safe database client with raw SQL support for pgvector queries
prisma/adapter-pg	PostgreSQL driver adapter required by Prisma 7
node-fetch	HTTP client for backend-to-AI-service calls
multer	Multipart form handling for image uploads
supabase/supabase-js	Supabase client for storage operations and Auth verification
google/generative-ai	Google Gemini SDK for the LLM insight agent
anthropic-ai/sdk	Anthropic Claude SDK (fallback LLM provider)
dotenvx	Environment variable management

Database & Storage

Technology	Purpose
PostgreSQL	Primary relational database (hosted on Supabase, aws-eu-central-1 region)
pgvector (PostgreSQL extension)	Stores 512-dimensional CLIP embeddings per product image; enables cosine distance queries via the <=> operator
Supabase	Managed PostgreSQL host + Storage + Auth platform
Supabase Storage	Object storage for three buckets: raw-images (phone photos), processed-images (AI-enhanced catalog images), vto-results (Virtual Try-On output)

AI & Machine Learning

Technology	Purpose
BiRefNet (ZhengPeng7/BiRefNet, HuggingFace)	Salient object segmentation for background removal , outperforms rembg on clothing items
CLIP (openai/clip-vit-base-patch32)	Vision-Language model producing 512-dim normalized embeddings for both images and text queries; enables cross-modal similarity search
YOLOS (valentinafeve/yolos-fashionpedia)	Fine-tuned clothing item detection on Fashionpedia dataset
FitDiT (diffusion model)	Virtual Try-On , generates photorealistic images of garments on virtual models
SAM2 (Segment Anything Model 2)	Garment segmentation mask generation required by the FitDiT pipeline
Python 3.11 + FastAPI	AI microservice runtime; exposes /process, /embed, /embed-text, /detect endpoints
PyTorch	Deep learning inference backend for BiRefNet, CLIP, and YOLOS
Pillow / NumPy	Image preprocessing: cropping, HSV color analysis, canvas normalization
deep-translator	Fallback Hebrew to English translation for unknown words in text search queries
Google Gemini API	Default LLM provider for the autonomous insight agent (tool-use loop with 7 store-data tools)
Anthropic Claude API	Optional LLM fallback for the insight agent

Authentication

Technology	Purpose
Supabase Auth	Admin authentication via email + password
JWT (JSON Web Token)	Session tokens verified on every admin API request via supabase.auth.getUser()

Infrastructure & DevOps

Technology	*Purpose*
Docker	*Containerizes the backend (port 5000) and AI service (port 8001); images built locally and run via docker run --env-file*
GitHub Actions	*CI/CD pipeline: builds and pushes the VTO RunPod Docker image to GitHub Container Registry (GHCR) on every push to main*
GitHub Container Registry (GHCR)	*Hosts the FitDiT + SAM2 VTO Docker image pulled by RunPod on job execution*
RunPod (Serverless GPU)	*On-demand GPU inference for Virtual Try-On jobs; cold-start model loading with job polling via REST API*
Supabase (Platform)	*Hosted PostgreSQL + Storage + Auth , eliminates the need for self-managed database infrastructure*